第 6 章

支持向量机*

山东大学数学学院

2023 数据科学

崔添†

目录

6.1 间隔与支持向量 **2**

6.2 对偶问题 **3**

6.2.1 拉格朗日乘子法 4

6.2.2 二次规划 6

6.2.3 支持向量机的对偶问题求解 6

6.3 核函数 **9**

6.3.1 核函数与 Mercer 定理 9

6.3.2 矩阵论复习 12

6.3.3 核函数常用性质 13

6.4 软间隔与正则化 **14**

6.4.1 软间隔 15

6.4.2 正则化 18

6.5 支持向量回归 **19**

6.6 核方法 **22**

6.6.1 表示定理 23

6.6.2 KLDA 示例 23

*注：本讲义基于周志华《机器学习》（西瓜书）整理，供“多元统计分析”课程使用；公式索引与原书保持一致。

†邮箱：ct@mail.sdu.edu.cn；GitHub：github.com/Tristan-sdu

6.1 间隔与支持向量

给定训练样本集 $D=\{(\boldsymbol{x}_1,y_1),(\boldsymbol{x}_2,y_2),\ldots,(\boldsymbol{x}_m,y_m)\},y_i\in\{-1,+1\}$，分类学习最基本的想法就是基于训练集 D 在样本空间中找到一个划分超平面，将不同类别的样本分开。但能将训练样本分开的划分超平面可能有很多，如图 6.1所示，我们应该努力去找到哪一个呢？

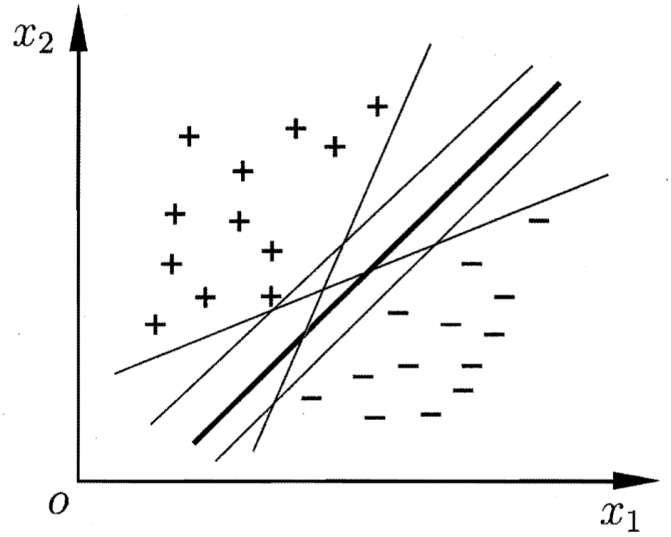


图 6.1 存在多个划分超平面将两类训练样本分开

直观上看，应该去找位于两类训练样本“正中间”的划分超平面，即图 6.1中红色的那个，因为该划分超平面对训练样本局部扰动的“容忍”性最好。例如，由于训练集的局限性或噪声的因素，训练集外的样本可能比图 6.1中的训练样本更接近两个类的分隔界，这将使许多划分超平面出现错误，而红色的超平面受影响最小。换言之，这个划分超平面所产生的分类结果是最鲁棒的，对未见示例的泛化能力最强。

在样本空间中，划分超平面可通过如下线性方程来描述：

$$\boldsymbol{w}^T\boldsymbol{x}+b=0 \tag{6.1}$$

其中 $\boldsymbol{w}=(w_1;w_2;\ldots;w_d)$ 为法向量，决定了超平面的方向；b 为位移项，决定了超平面与原点之间的距离。显然，划分超平面可被法向量 $\boldsymbol{w}$ 和位移 b 确定，下面我们将其记为 $(\boldsymbol{w},b)$。样本空间中任意点 $\boldsymbol{x}$ 到超平面 $(\boldsymbol{w},b)$ 的距离可写为：

$$r=\frac{|\boldsymbol{w}^T\boldsymbol{x}+b|}{||\boldsymbol{w}||} \tag{6.2}$$

假设超平面 $(\boldsymbol{w},b)$ 能将训练样本正确分类，即对于 $(\boldsymbol{x}_i,y_i)\in D$，若 $y_i=+1$，则有 $\boldsymbol{w}^T\boldsymbol{x}_i+b>0$；若 $y_i=-1$，则有 $\boldsymbol{w}^T\boldsymbol{x}_i+b<0$。令

$$\begin{cases}\boldsymbol{w}^T\boldsymbol{x}_i+b\geq+1, & y_i=+1\\ \boldsymbol{w}^T\boldsymbol{x}_i+b\leq-1, & y_i=-1\end{cases} \tag{6.3}$$

如图 6.2所示，距离超平面最近的这几个训练样本点使式(6.3)的等号成立，它们被称为“支持向量”(support vector)，两个异类支持向量到超平面的距离之和为

$$\gamma = \frac{2}{||\boldsymbol{w}||} \tag{6.4}$$

它被称为“间隔”(margin)。

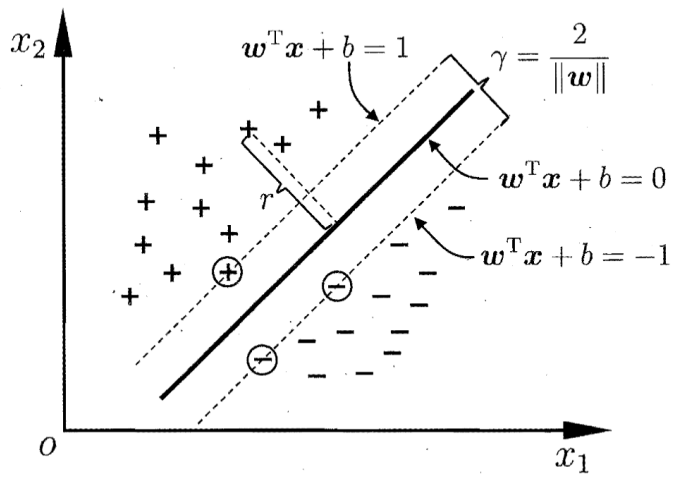


图 6.2 支持向量与间隔

欲找到具有“最大间隔”(maximum margin) 的划分超平面，也就是要找到能满足式(6.3)中约束的参数 $\boldsymbol{w}$ 和 b，使得 γ 最大，即

$$\max_{\boldsymbol{w},b} \frac{2}{||\boldsymbol{w}||} \tag{6.5}$$

$$\text{s.t. } y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1, \quad i = 1, 2, \ldots, m.$$

显然，为了最大化间隔，仅需最大化 $||\boldsymbol{w}||^{-1}$，这等价于最小化 $||\boldsymbol{w}||^2$。于是，式(6.5)可重写为

$$\min_{\boldsymbol{w},b} \frac{1}{2}||\boldsymbol{w}||^2 \tag{6.6}$$

$$\text{s.t. } y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1, \quad i = 1, 2, \ldots, m.$$

这就是支持向量机 (Support Vector Machine，简称 SVM) 的基本型。

6.2 对偶问题

为了更好地理解和解决式(6.6)这一支持向量机基本型的优化问题，我们首先需要补充一些数学优化方面的基础知识，即拉格朗日乘子法与二次规划。

6.2.1 拉格朗日乘子法

拉格朗日乘子法 (Lagrange multipliers) 是一种寻找多元函数在一组约束下的极值的方法。通过引入拉格朗日乘子，可将有 d 个变量与 k 个约束条件的最优化问题转化为具有 $d+k$ 个变量的无约束优化问题求解。

先考虑一个等式约束的优化问题。假定 $\boldsymbol{x}$ 为 d 维向量，欲寻找 $\boldsymbol{x}$ 的某个取值 $\boldsymbol{x}^*$，使目标函数 $f(\boldsymbol{x})$ 最小且同时满足 $g(\boldsymbol{x})=0$ 的约束。从几何角度看，该问题的目标是在由方程 $g(\boldsymbol{x})=0$ 确定的 $d-1$ 维曲面上寻找能使目标函数 $f(\boldsymbol{x})$ 最小化的点。此时不难得到如下结论：

- 对于约束曲面上的任意点 $\boldsymbol{x}$，该点的梯度 $\nabla g(\boldsymbol{x})$ 正交于约束曲面。
- 在最优点 $\boldsymbol{x}^*$，目标函数在该点的梯度 $\nabla f(\boldsymbol{x}^*)$ 正交于约束曲面。

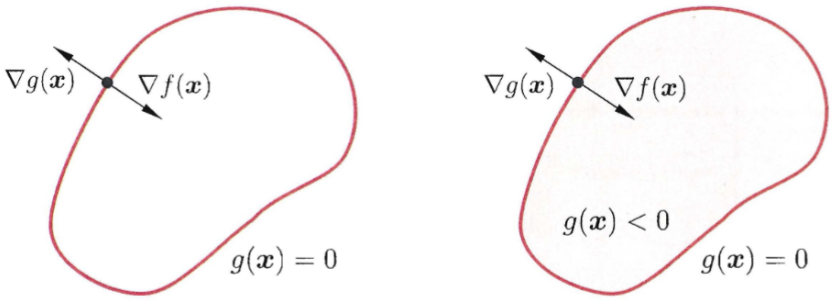
由此可知，在最优点 $\boldsymbol{x}^*$，梯度 $\nabla g(\boldsymbol{x}^*)$ 和 $\nabla f(\boldsymbol{x}^*)$ 的方向必相同或相反，即存在 $\lambda \neq 0$ 使得

$$\nabla f(\boldsymbol{x}^*) + \lambda \nabla g(\boldsymbol{x}^*) = 0 \tag{B.1}$$

λ 称为拉格朗日乘子。定义拉格朗日函数

$$L(\boldsymbol{x}, \lambda) = f(\boldsymbol{x}) + \lambda g(\boldsymbol{x}) \tag{B.2}$$

不难发现，将其对 $\boldsymbol{x}$ 的偏导数 $\nabla_{\boldsymbol{x}} L(\boldsymbol{x}, \lambda)$ 置零即得式 (B.1)，同时，将其对 λ 的偏导数 $\nabla_{\lambda} L(\boldsymbol{x}, \lambda)$ 置零即得约束条件 $g(\boldsymbol{x})=0$。于是，原约束优化问题可转化为对拉格朗日函数 $L(\boldsymbol{x}, \lambda)$ 的无约束优化问题。



(a) 等式约束　　(b) 不等式约束

现在考虑不等式约束 $g(\boldsymbol{x}) \leq 0$。此时最优点 $\boldsymbol{x}^*$ 或在 $g(\boldsymbol{x}) < 0$ 的区域中，或在边界 $g(\boldsymbol{x})=0$ 上。对于 $g(\boldsymbol{x})<0$ 的情形，约束不起作用，可直接通过条件 $\nabla f(\boldsymbol{x})=0$ 来获得最优点；这等价于将 λ 置零然后对 $\nabla_{\boldsymbol{x}} L(\boldsymbol{x}, \lambda)$ 置零得到最优点。对于 $g(\boldsymbol{x})=0$ 的情形类似于上面等式约束，但需注意的是，此时 $\nabla f(\boldsymbol{x}^*)$ 的方向必与 $\nabla g(\boldsymbol{x}^*)$ 相反，即存在常数 $\lambda > 0$ 使得 $\nabla f(\boldsymbol{x}^*) + \lambda \nabla g(\boldsymbol{x}^*) = 0$。整合这两种情形，必满足 $\lambda g(\boldsymbol{x}) = 0$。

因此，在约束 $g(\boldsymbol{x}) \leq 0$ 下最小化 $f(\boldsymbol{x})$，可转化为在如下约束下最小化式 (B.2) 的拉格朗日函数：

$$\begin{cases} g(\boldsymbol{x}) \leq 0; \\ \lambda \geq 0; \\ \lambda g(\boldsymbol{x}) = 0. \end{cases} \tag{B.3}$$

式 (B.3) 称为 Karush-Kuhn-Tucker(KKT) 条件。

上述做法可推广到多个约束。考虑具有 m 个等式约束和 n 个不等式约束，且可行域 $D \subset \mathbb{R}^d$ 非空的优化问题

$$\begin{aligned} &\min_{\boldsymbol{x}} f(\boldsymbol{x}) \\ &\text{s.t. } h_i(\boldsymbol{x}) = 0 \quad (i = 1, 2, \ldots, m), \\ &g_j(\boldsymbol{x}) \leq 0 \quad (j = 1, 2, \ldots, n). \end{aligned} \tag{B.4}$$

引入拉格朗日乘子 $\boldsymbol{\lambda} = (\lambda_1, \lambda_2, \ldots, \lambda_m)^T$ 和 $\boldsymbol{\mu} = (\mu_1, \mu_2, \ldots, \mu_n)^T$，相应的拉格朗日函数为

$$L(\boldsymbol{x}, \boldsymbol{\lambda}, \boldsymbol{\mu}) = f(\boldsymbol{x}) + \sum_{i=1}^{m} \lambda_i h_i(\boldsymbol{x}) + \sum_{j=1}^{n} \mu_j g_j(\boldsymbol{x}). \tag{B.5}$$

由不等式约束引入的 KKT 条件 $(j = 1, 2, \ldots, n)$ 为

$$\begin{cases} g_j(\boldsymbol{x}) \leq 0; \\ \mu_j \geq 0; \\ \mu_j g_j(\boldsymbol{x}) = 0. \end{cases} \tag{B.6}$$

一个优化问题可以从两个角度来考察，即“主问题”(primal problem) 和“对偶问题”(dual problem)。对主问题而言，基于拉格朗日函数，其拉格朗日“对偶函数”(dual function) $\Gamma : \mathbb{R}^m \times \mathbb{R}^n \mapsto \mathbb{R}$ 定义为

$$\Gamma(\boldsymbol{\lambda}, \boldsymbol{\mu}) = \inf_{\boldsymbol{x} \in D} L(\boldsymbol{x}, \boldsymbol{\lambda}, \boldsymbol{\mu}). \tag{B.7}$$

若 $\tilde{\boldsymbol{x}} \in D$ 为主问题可行域中的点，则对任意 $\boldsymbol{\mu} \geq 0$ 和 $\boldsymbol{\lambda}$ 都有

$$\sum_{i=1}^{m} \lambda_i h_i(\tilde{\boldsymbol{x}}) + \sum_{j=1}^{n} \mu_j g_j(\tilde{\boldsymbol{x}}) \leq 0, \tag{B.8}$$

进而有

$$\Gamma(\boldsymbol{\lambda}, \boldsymbol{\mu}) = \inf_{\boldsymbol{x} \in D} L(\boldsymbol{x}, \boldsymbol{\lambda}, \boldsymbol{\mu}) \leq L(\tilde{\boldsymbol{x}}, \boldsymbol{\lambda}, \boldsymbol{\mu}) \leq f(\tilde{\boldsymbol{x}}). \tag{B.9}$$

若主问题的最优值为 p^*，则对任意 $\boldsymbol{\mu} \geq 0$ 和 $\boldsymbol{\lambda}$ 都有

$$\Gamma(\boldsymbol{\lambda}, \boldsymbol{\mu}) \leq p^*. \tag{B.10}$$

即对偶函数给出了主问题最优值的下界。显然，这个下界取决于 $\boldsymbol{\mu}$ 和 $\boldsymbol{\lambda}$ 的值。于是，一个很自然的问题是：基于对偶函数能获得的最好下界是什么？这就引出了优化问题

$$\max_{\boldsymbol{\lambda},\boldsymbol{\mu}} \Gamma(\boldsymbol{\lambda},\boldsymbol{\mu}) \quad \text{s.t. } \boldsymbol{\mu} \geq 0. \tag{B.11}$$

式 (B.11) 就是主问题 (B.4) 的对偶问题，其中 $\boldsymbol{\lambda}$ 和 $\boldsymbol{\mu}$ 称为“对偶变量”(dual variable)。无论主问题 (B.4) 的凸性如何，对偶问题 (B.11) 始终是凸优化问题。

考虑对偶问题的最优值 d^*，显然有 $d^* \leq p^*$，这称为“弱对偶性”(weak duality) 成立；若 $d^* = p^*$，则称为“强对偶性”(strong duality) 成立，此时由对偶问题能获得主问题的最优下界。对于一般优化问题，强对偶性通常不成立。但是，若主问题为凸优化问题，如 $f(\boldsymbol{x})$ 和 $g_j(\boldsymbol{x})$ 均为凸函数，$h_i(\boldsymbol{x})$ 为仿射函数，且其可行域中至少有一点使不等式约束严格成立，则此时强对偶性成立。值得注意的是，在强对偶性成立时，将拉格朗日函数分别对原变量和对偶变量求导，再并令导数等于零，即可得到原变量与对偶变量的数值关系。于是，对偶问题解决了，主问题也就解决了。

6.2.2 二次规划

二次规划 (Quadratic Programming，简称 QP) 是一类典型的优化问题，包括凸二次优化和非凸二次优化。在此类问题中，目标函数是变量的二次函数，而约束条件是变量的线性不等式。假定变量个数为 d，约束条件的个数为 m，则标准的二次规划问题形如：

$$\begin{aligned} \min_{\boldsymbol{x}} \ & \frac{1}{2}\boldsymbol{x}^T\mathbf{Q}\boldsymbol{x} + \boldsymbol{c}^T\boldsymbol{x} \\ \text{s.t. } & \mathbf{A}\boldsymbol{x} \leq \boldsymbol{b} \end{aligned} \tag{B.12}$$

其中 $\boldsymbol{x}$ 为 d 维向量，$\mathbf{Q} \in \mathbb{R}^{d\times d}$ 为实对称矩阵，$\mathbf{A} \in \mathbb{R}^{m\times d}$ 为实矩阵，$\boldsymbol{b} \in \mathbb{R}^m$ 和 $\boldsymbol{c} \in \mathbb{R}^d$ 为实向量。

非标准二次规划问题中可以包含等式约束。注意到等式约束能用两个不等式约束来代替；不等式约束也可通过增加松弛变量的方式转化为等式约束。

若 $\mathbf{Q}$ 为半正定矩阵，则目标函数是凸函数，相应的二次规划是凸二次优化问题；此时若约束条件定义的可行域不为空，且目标函数在此可行域有下界，则该问题将有全局最小值。若 $\mathbf{Q}$ 为正定矩阵，则该问题有唯一的全局最小值。若 $\mathbf{Q}$ 为非正定矩阵，则相应问题是有多个平稳点和局部极小点的 NP 难问题。

常用的二次规划解法有椭球法 (ellipsoid method)、内点法 (interior point method)、增广拉格朗日法 (augmented Lagrangian)、梯度投影法 (gradient projection) 等。支持向量机的基本型及其对偶问题便是一个典型的凸二次优化问题。

6.2.3 支持向量机的对偶问题求解

我们希望求解式(6.6)来得到大间隔划分超平面所对应的模型

$$f(\boldsymbol{x}) = \boldsymbol{w}^T\boldsymbol{x} + b \tag{6.7}$$

其中 $\boldsymbol{w}$ 和 b 是模型参数。注意到式(6.6)本身是一个凸二次规划 (convex quadratic programming) 问题，能直接用现成的优化计算包求解，但我们可以有更高效的办法。

对式(6.6)使用拉格朗日乘子法可得到其“对偶问题”(dual problem)。具体来说，对式(6.6)的每条约束添加拉格朗日乘子 $\alpha_i \geq 0$，则该问题的拉格朗日函数可写为

$$L(\boldsymbol{w}, b, \boldsymbol{\alpha}) = \frac{1}{2}||\boldsymbol{w}||^2 + \sum_{i=1}^{m} \alpha_i (1 - y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b)) \tag{6.8}$$

其中 $\boldsymbol{\alpha} = (\alpha_1; \alpha_2; \ldots; \alpha_m)$。令 $L(\boldsymbol{w}, b, \boldsymbol{\alpha})$ 对 $\boldsymbol{w}$ 和 b 的偏导为零可得

$$\boldsymbol{w} = \sum_{i=1}^{m} \alpha_i y_i \boldsymbol{x}_i \tag{6.9}$$

$$0 = \sum_{i=1}^{m} \alpha_i y_i \tag{6.10}$$

将式(6.9)代入式(6.8)，即可将 $L(\boldsymbol{w}, b, \boldsymbol{\alpha})$ 中的 $\boldsymbol{w}$ 和 b 消去，再考虑式(6.10)的约束，就得到式(6.6)的对偶问题

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha_i \alpha_j y_i y_j \boldsymbol{x}_i^T \boldsymbol{x}_j \tag{6.11}$$

$$\text{s.t.} \quad \sum_{i=1}^{m} \alpha_i y_i = 0, \quad \alpha_i \geq 0, \quad i = 1, 2, \ldots, m.$$

解出 $\boldsymbol{\alpha}$ 后，求出 $\boldsymbol{w}$ 与 b 即可得到模型

$$f(\boldsymbol{x}) = \boldsymbol{w}^T \boldsymbol{x} + b = \sum_{i=1}^{m} \alpha_i y_i \boldsymbol{x}_i^T \boldsymbol{x} + b \tag{6.12}$$

从对偶问题(6.11)解出的 α_i 是式(6.8)中的拉格朗日乘子，它恰对应着训练样本 $(\boldsymbol{x}_i, y_i)$。注意到式(6.6)中有不等式约束，因此上述过程需满足 KKT(Karush-Kuhn-Tucker) 条件，即要求

$$\begin{cases} \alpha_i \geq 0; \\ y_i f(\boldsymbol{x}_i) - 1 \geq 0; \\ \alpha_i (y_i f(\boldsymbol{x}_i) - 1) = 0. \end{cases} \tag{6.13}$$

于是，对任意训练样本 $(\boldsymbol{x}_i, y_i)$，总有 $\alpha_i = 0$ 或 $y_i f(\boldsymbol{x}_i) = 1$。若 $\alpha_i = 0$，则该样本将不会在式(6.12)的求和中出现，也就不会对 $f(\boldsymbol{x})$ 有任何影响；若 $\alpha_i > 0$ 则必有 $y_i f(\boldsymbol{x}_i) = 1$，所对应的样本点位于最大间隔边界上，是一个支持向量。这显示出支持向量机的一个重要性质：训练完成后，大部分的训练样本都不需保留，最终模型仅与支持向量有关。

那么，如何求解式(6.11)呢？不难发现，这是一个二次规划问题，可使用通用的二次规划算法来求解；然而，该问题的规模正比于训练样本数，这会在实际任务中造成很大的开销。为了避开这个障碍，人们通过利用问题本身的特性，提出了很多高效算法，SMO(Sequential Minimal Optimization) 是其中一个著名的代表。

SMO 的基本思路是先固定 α_i 之外的所有参数，然后求 α_i 上的极值。由于存在约束 $\sum_{i=1}^m \alpha_i y_i = 0$，若固定 α_i 之外的其他变量，则 α_i 可由其他变量导出。于是，SMO 每次选择两个变量 α_i 和 α_j，并固定其他参数。这样，在参数初始化后，SMO 不断执行如下两个步骤直至收敛：

- 选取一对需更新的变量 α_i 和 α_j；
- 固定 α_i 和 α_j 以外的参数，求解式(6.11)获得更新后的 α_i 和 α_j。

注意到只需选取的 α_i 和 α_j 中有一个不满足 KKT 条件(6.13)，目标函数就会在迭代后增大。直观来看，KKT 条件违背的程度越大，则变量更新后可能导致的目标函数值增幅越大。于是，SMO 先选取违背 KKT 条件程度最大的变量。第二个变量应选择一个使目标函数值增长最快的变量，但由于比较各变量所对应的目标函数值增幅的复杂度过高，因此 SMO 采用了一个启发式：使选取的两变量所对应样本之间的间隔最大。一种直观的解释是，这样的两个变量有很大的差别，与对两个相似的变量进行更新相比，对它们进行更新会带给目标函数值更大的变化。

SMO 算法之所以高效，恰由于在固定其他参数后，仅优化两个参数的过程能做到非常高效。具体来说，仅考虑 α_i 和 α_j 时，式(6.11)中的约束可重写为

$$\alpha_i y_i + \alpha_j y_j = c, \quad \alpha_i \geq 0, \quad \alpha_j \geq 0 \tag{6.14}$$

其中

$$c = -\sum_{k \neq i,j} \alpha_k y_k \tag{6.15}$$

是使 $\sum_{i=1}^m \alpha_i y_i = 0$ 成立的常数。用

$$\alpha_i y_i + \alpha_j y_j = c \tag{6.16}$$

消去式(6.11)中的变量 α_j，则得到一个关于 α_i 的单变量二次规划问题，仅有的约束是 $\alpha_i \geq 0$。不难发现，这样的二次规划问题具有闭式解，于是不必调用数值优化算法即可高效地计算出更新后的 α_i 和 α_j。

【注记：如何形象地理解 SMO 算法?】

为了更直观地理解，我们可以把 SMO 算法想象成在操控一个有一万个滑动推子（对应拉格朗日乘子 α_1 到 α_m）的巨型调音台，目标是寻找最优音质（也就是使目标函数最大化）。但是，调音台上有一个硬性的连杆规则，要求所有推子乘以各自的加权标签必须永远保持平衡，总和为 0（即等式约束 $\sum \alpha_i y_i = 0$）。

如果你想调音，只单独动其中一个推子，总和立刻就不等于 0 了，规则不允许。所以，你能采取的符合规则的**最小动作（Minimal）**，就是同时抓住**两根**推子。由于只动两根，它们的相对运动是被完全锁死的：你把左手推子往上推一点，右手推子就必须相应地拉下来一点。

整个 SMO 算法其实就是在不断重复以下动作：

1. 挑出当时状态最不和谐的一根推子（也就是严重违背 KKT 条件的变量 α_i）。
2. 选出另一根和它搭配起来能使得音质变化最大的推子 α_j。
3. **将另外 99998 根推子全部锁死固定，视为常数。**
4. 此时只剩下这两个彼此锁喉的推子，问题极度简化，不需要使用任何笨重的矩阵求解器，直接用类似初中极值公式的闭式解析解，在一瞬间就能将这两根推子推到当前切面下的绝对最优位置。
5. 如此不断循环（Sequential）接力，直到所有推子都找准了符合 KKT 条件的完美刻度。

借由这种“化巨集为散点”和“降维打击”的智慧，SMO 将海量参数的二次矩阵规划难题，降级成了解无数个初等二元小方程，使得具有成千上万样本量的大型支持向量机训练成为可能。

如何确定偏移项 b 呢？注意到对任意支持向量 $(\boldsymbol{x}_s, y_s)$ 都有 $y_s f(\boldsymbol{x}_s) = 1$，即

$$y_s \left(\sum_{i=1}^{m} \alpha_i y_i \boldsymbol{x}_i^T \boldsymbol{x}_s + b \right) = 1 \tag{6.17}$$

其中 $S = \{i \mid \alpha_i > 0, i = 1, 2, \ldots, m\}$ 为所有支持向量的下标集。理论上，可选取任意支持向量并通过求解式(6.17)获得 b，但现实任务中常采用一种更鲁棒的做法：使用所有支持向量求解的平均值

$$b = \frac{1}{|S|} \sum_{s \in S} \left(y_s - \sum_{i=1}^{m} \alpha_i y_i \boldsymbol{x}_i^T \boldsymbol{x}_s \right) \tag{6.18}$$

6.3 核函数

6.3.1 核函数与 Mercer 定理

在本章前面的讨论中，我们假设训练样本是线性可分的，即存在一个划分超平面能将训练样本正确分类。然而在现实任务中，原始样本空间内也许并不存在一个能正确划分两类样本的超平面。例如图 6.3中的“异或”问题就不是线性可分的。

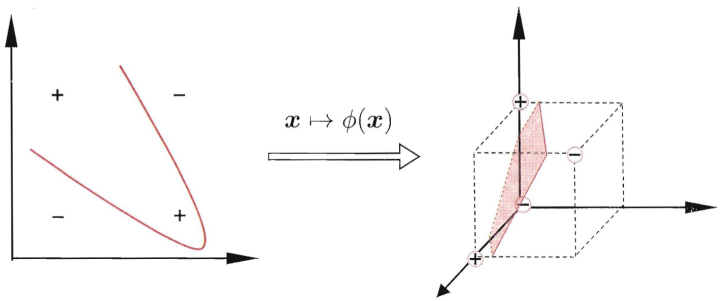


图 6.3 异或问题与非线性映射

对这样的问题，可将样本从原始空间映射到一个更高维的特征空间，使得样本在这个特征空间内线性可分。例如在图 6.3中，若将原始的二维空间映射到一个合适的三维空间，就能找到一个合适的划分超平面。幸运的是，如果原始空间是有限维，即属性数有限，那么一定存在一个高维特征空间使样本可分。

令 $\phi(\boldsymbol{x})$ 表示将 $\boldsymbol{x}$ 映射后的特征向量，于是，在特征空间中划分超平面所对应的模型可表示为

$$f(\boldsymbol{x}) = \boldsymbol{w}^T\phi(\boldsymbol{x}) + b \tag{6.19}$$

其中 $\boldsymbol{w}$ 和 b 是模型参数。类似式(6.6)，有

$$\min_{\boldsymbol{w},b} \frac{1}{2}||\boldsymbol{w}||^2 \tag{6.20}$$

$$\text{s.t. } y_i(\boldsymbol{w}^T\phi(\boldsymbol{x}_i) + b) \geq 1, \quad i = 1, 2, \ldots, m.$$

其对偶问题是

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{m} \alpha_i - \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{m} \alpha_i\alpha_j y_i y_j \phi(\boldsymbol{x}_i)^T\phi(\boldsymbol{x}_j) \tag{6.21}$$

$$\text{s.t. } \sum_{i=1}^{m} \alpha_i y_i = 0, \quad \alpha_i \geq 0, \quad i = 1, 2, \ldots, m.$$

求解式(6.21)涉及到计算 $\phi(\boldsymbol{x}_i)^T\phi(\boldsymbol{x}_j)$，这是样本 $\boldsymbol{x}_i$ 与 $\boldsymbol{x}_j$ 映射到特征空间之后的内积。由于特征空间维数可能很高，甚至可能是无穷维，因此直接计算 $\phi(\boldsymbol{x}_i)^T\phi(\boldsymbol{x}_j)$ 通常是困难的。为了避开这个障碍，可以设想这样一个函数：

$$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \langle \phi(\boldsymbol{x}_i), \phi(\boldsymbol{x}_j) \rangle = \phi(\boldsymbol{x}_i)^T\phi(\boldsymbol{x}_j) \tag{6.22}$$

即 $\boldsymbol{x}_i$ 与 $\boldsymbol{x}_j$ 在特征空间的内积等于它们在原始样本空间中通过函数 $\kappa(\cdot,\cdot)$ 计算的结果。有了这样的函数我们就不必直接去计算高维甚至无穷维特征空间中的内积。

【注记：核函数的形象化理解】
想象桌面上红蓝弹珠混合在一起，无法用一把直尺（线性分类器）分开。如果我们“拍一下桌子”，让弹珠弹到空中（映射到高维空间），我们就能在半空中插进一张平滑的纸（超平面）将其完美分离。
但如果弹珠数量巨大，计算每颗弹珠弹跳轨迹（即高维映射 $\phi(\cdot)$）的代价无法想象。而**核函数**就像一个“魔法按钮”：它不需要真的模拟弹珠在空中的轨迹，只需要输入地上两颗弹珠的原始坐标，就能直接吐出它们如果在空中碰撞（高维内积）后的结果。
简单来说：映射是解决问题的**思路**（跳出低维），而核函数则是解决问题的**捷径**（身在低维，却能直接获取高维运算的利益，避开维度灾难）。

于是式(6.21)可重写为

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha_i \alpha_j y_i y_j \kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) \tag{6.23}$$

求解后即可得到

$$f(\boldsymbol{x}) = \boldsymbol{w}^T \phi(\boldsymbol{x}) + b = \sum_{i=1}^{m} \alpha_i y_i \kappa(\boldsymbol{x}, \boldsymbol{x}_i) + b \tag{6.24}$$

这里的函数 $\kappa(\cdot,\cdot)$ 就是“核函数”(kernel function)。式(6.24)显示出模型最优解可通过训练样本的核函数展开，这一展式亦称“支持向量展式”(support vector expansion)。

常见的核函数如表 6.1 所示。那么，什么样的函数能做核函数呢？我们有著名的 **Mercer 定理：**

定理 6.1 (Mercer 定理) 令 $\mathcal{X}$ 为输入空间，$\kappa(\cdot,\cdot)$ 是定义在 $\mathcal{X} \times \mathcal{X}$ 上的对称函数，则 κ 是核函数当且仅当对于任意数据 $D = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_m\}$，“核矩阵”(kernel matrix) K 总是半正定的：

$$K = \begin{bmatrix} \kappa(\boldsymbol{x}_1, \boldsymbol{x}_1) & \ldots & \kappa(\boldsymbol{x}_1, \boldsymbol{x}_m) \\ \vdots & \ddots & \vdots \\ \kappa(\boldsymbol{x}_m, \boldsymbol{x}_1) & \ldots & \kappa(\boldsymbol{x}_m, \boldsymbol{x}_m) \end{bmatrix}$$

定理 6.1 表明，只要一个对称函数所对应的核矩阵半正定，它就能作为核函数使用。事实上，对于一个半正定核矩阵，总能找到一个与之对应的映射 ϕ。换言之，任何一个核函数都隐式地定义了一个称为“再生核希尔伯特空间”(Reproducing Kernel Hilbert Space，简称 RKHS) 的特征空间。

通过前面的讨论可知，我们希望样本在特征空间内线性可分，因此特征空间的好坏对支持向量机的性能至关重要。需注意的是，在不知道特征映射的形式时，我们并不知道什么样的核函数是合适的，而核函数也仅是隐式地定义了这个特征空间。于是，“核函数选择”成为支持向量机的最大变数。若核函数选择不合适，则意味着将样本映射到了一个不合适的特征空间，很可能导致性能不佳。

6.3.2 矩阵论复习

由于 Mercer 定理是通过“核矩阵半正定”来刻画核函数的，因此这里先回顾若干与核函数密切相关的矩阵论概念与性质。需说明的是，在本讲义的机器学习语境下，默认讨论的矩阵均为实矩阵。

设 $A \in \mathbb{R}^{n\times n}$ 为**实对称矩阵**，若对任意 $\boldsymbol{x} \in \mathbb{R}^n$ 都有

$$\boldsymbol{x}^T A\boldsymbol{x} \geq 0$$

则称 A 为**半正定矩阵** (positive semidefinite matrix)，记作 $A \succeq 0$。

之所以在定义中强调“对称”，是因为对任意实矩阵 A 都有分解

$$A = \frac{A + A^T}{2} + \frac{A - A^T}{2},$$

其中第一项是对称矩阵，第二项是反对称矩阵。注意到对任意 $\boldsymbol{x}$，

$$\boldsymbol{x}^T \left(\frac{A - A^T}{2}\right) \boldsymbol{x} = 0,$$

因此二次型 $\boldsymbol{x}^T A\boldsymbol{x}$ 实际上只由 A 的对称部分决定。正因如此，在机器学习、优化和核方法中谈论半正定矩阵时，通常默认它是实对称矩阵。

注记：仅由对任意 $\boldsymbol{x}$ 都有 $\boldsymbol{x}^T A\boldsymbol{x} \geq 0$，并不能反推出 A 本身一定是实对称矩阵；严格地说，这个条件只能推出 A 的对称部分是半正定的。

对半正定矩阵，下面几个刻画最为常用：

- **特征值刻画：** $A \succeq 0$ 当且仅当 A 的全部特征值均非负；
- **Gram 表示：** $A \succeq 0$ 当且仅当存在一组向量 $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n$，使得

$$A_{ij} = \langle \boldsymbol{v}_i, \boldsymbol{v}_j \rangle;$$

- **分解形式：** $A \succeq 0$ 当且仅当存在矩阵 B 使得

$$A = B^T B.$$

其中，Gram 表示尤其重要，因为它直接揭示了核矩阵的本质：若矩阵 K 半正定，则它总可以看作某组向量两两内积构成的矩阵；这正对应于核函数

$$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \phi(\boldsymbol{x}_i)^T \phi(\boldsymbol{x}_j)$$

的形式。因此，从矩阵角度看，Mercer 定理实际上是在说明：**核函数的充要条件，就是它在任意样本点集上诱导出的 Gram 矩阵总是半正定。**

半正定矩阵还有若干重要闭包性质，它们恰好对应于后面构造新核函数时常用的规则：

- 若 $A \succeq 0, B \succeq 0$，且 $\alpha, \beta \geq 0$，则 $\alpha A + \beta B \succeq 0$；
- 若 $A \succeq 0$，则对任意矩阵 G，合同变换

$$GAG^T$$

  仍是半正定矩阵；
- 若 $A \succeq 0, B \succeq 0$，则它们的 Hadamard 积 $A \circ B$ 仍为半正定矩阵；
- 若 $A \succeq 0$，则 A 的任意主子矩阵仍为半正定矩阵。

这些性质在核方法中反复出现。例如，核函数的非负线性组合对应于核矩阵的非负线性组合；核函数的逐点乘积对应于核矩阵的 Hadamard 积；而式(6.27)所对应的“函数缩放”则可理解为核矩阵在对角矩阵作用下的合同变换。

总的来说，半正定矩阵可以看作“内积结构”的矩阵化表达；而核函数则是把这种内积结构从有限维向量空间推广到一般输入空间中的工具。这正是矩阵论与 Mercer 定理、核函数之间的内在联系。

6.3.3 核函数常用性质

表 6.1 常用核函数与特征空间维度

名称	表达式	参数	映射空间维度
线性核	$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \boldsymbol{x}_i^T \boldsymbol{x}_j$	无	原始空间维度 d
多项式核	$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = (\boldsymbol{x}_i^T \boldsymbol{x}_j)^d$	$d \geq 1$ 为多项式次数	高维（有限）
高斯核	$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \exp\left(-\frac{\lVert \boldsymbol{x}_i - \boldsymbol{x}_j \rVert^2}{2\sigma^2}\right)$	$\sigma > 0$ 为高斯核带宽	**无限维 (Infinite)**
拉普拉斯核	$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \exp\left(-\frac{\lVert \boldsymbol{x}_i - \boldsymbol{x}_j \rVert}{\sigma}\right)$	$\sigma > 0$	**无限维 (Infinite)**
Sigmoid 核	$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \tanh\left(\beta \boldsymbol{x}_i^T \boldsymbol{x}_j + \theta\right)$	$\beta > 0, \theta < 0$	**无限维 (Infinite)**

【深度思考：为什么高斯核对应无限维空间?】

我们可以通过泰勒展开（Taylor Expansion）直观地说明这一点。为简化推导，假设数据为一维 $x, z \in \mathbb{R}$，带宽 $\sigma = 1$：

1. 展开高斯核指数项：$\kappa(x,z) = \exp\left(-\frac{(x-z)^2}{2}\right) = \exp\left(-\frac{x^2+z^2-2xz}{2}\right) = e^{-\frac{x^2}{2}} e^{-\frac{z^2}{2}} e^{xz}$
2. 利用指数函数的泰勒展开式 $e^u = \sum_{n=0}^{\infty} \frac{u^n}{n!}$：$e^{xz} = \sum_{n=0}^{\infty} \frac{(xz)^n}{n!} = 1 + xz + \frac{(xz)^2}{2!} + \cdots + \frac{(xz)^n}{n!} + \ldots$
3. 将结果代回原式，可得 $\kappa(x,z) = e^{-\frac{x^2}{2}} e^{-\frac{z^2}{2}} \sum_{n=0}^{\infty} \frac{x^n z^n}{n!}$。若定义特征映射 $\phi(x) = e^{-\frac{x^2}{2}} \left[1, x, \frac{x^2}{\sqrt{2!}}, \ldots, \frac{x^n}{\sqrt{n!}}, \ldots\right]^T$，则有 $\kappa(x,z) = \phi(x)^T \phi(z)$。

由于映射向量包含自变量 x 的任意 n 次方项（直至无穷），因此其对应的特征空间是**无限维**的。

此外，还可通过函数组合得到，例如：

- 若 κ_1 和 κ_2 为核函数，则对于任意正数 γ_1, γ_2，其线性组合

$$\gamma_1 \kappa_1(\boldsymbol{x}_i, \boldsymbol{x}_j) + \gamma_2 \kappa_2(\boldsymbol{x}_i, \boldsymbol{x}_j) \tag{6.25}$$

  也是核函数；

- 若 κ_1 和 κ_2 为核函数，则核函数的直积

$$\kappa_1(\boldsymbol{x}_i, \boldsymbol{x}_j) \otimes \kappa_2(\boldsymbol{x}_i, \boldsymbol{x}_j) = \kappa_1(\boldsymbol{x}_i, \boldsymbol{x}_j) \kappa_2(\boldsymbol{x}_i, \boldsymbol{x}_j) \tag{6.26}$$

  也是核函数；

- 若 κ_1 为核函数，则其在任意函数 $g(\boldsymbol{x})$ 诱导下的合同变换

$$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = g(\boldsymbol{x}_i) \kappa_1(\boldsymbol{x}_i, \boldsymbol{x}_j) g(\boldsymbol{x}_j) \tag{6.27}$$

  也是核函数。

6.4 软间隔与正则化

在前面的讨论中，我们一直假定训练样本在样本空间或特征空间中是线性可分的，即存在一个超平面能将不同类的样本完全划分开。然而，在现实任务中往往很难确定合适的核函数使得训练样本在特征空间中线性可分；退一步说，即便恰好找到了某个核函数使训练集在特征空间中线性可分，也很难断定这个貌似线性可分的结果不是由于过拟合所造成的。

6.4.1 软间隔

缓解该问题的一个办法是允许支持向量机在一些样本上出错。为此，要引入“软间隔”(soft margin) 的概念，如图 6.4所示。

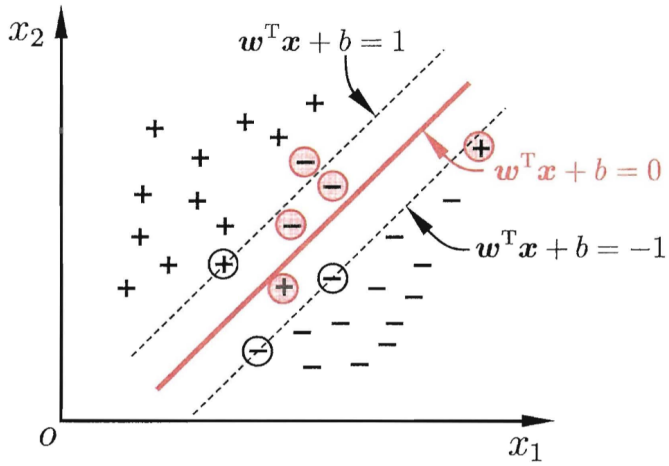


图 6.4 软间隔示意图。红色圈出了一些不满足约束的样本。

具体来说，前面介绍的支持向量机形式是要求所有样本均满足约束(6.3)，即所有样本都必须划分正确，这称为“硬间隔”(hard margin)，而软间隔则是允许某些样本不满足约束

$$y_i(\boldsymbol{w}^T\boldsymbol{x}_i+b)\geq 1 \tag{6.28}$$

当然，在最大化间隔的同时，不满足约束的样本应尽可能少。于是，优化目标可写为

$$\min_{\boldsymbol{w},b}\frac{1}{2}||\boldsymbol{w}||^2+C\sum_{i=1}^{m}\ell_{0/1}\left(y_i(\boldsymbol{w}^T\boldsymbol{x}_i+b)-1\right) \tag{6.29}$$

其中 $C>0$ 是一个常数，$\ell_{0/1}$ 是“0/1 损失函数”

$$\ell_{0/1}(z)=\begin{cases}1, & \text{if } z<0;\\ 0, & \text{otherwise.}\end{cases} \tag{6.30}$$

显然，当 C 为无穷大时，式(6.29)迫使所有样本均满足约束(6.28)，于是式(6.29)等价于(6.6)；当 C 取有限值时，式(6.29)允许一些样本不满足约束。

然而，$\ell_{0/1}$ 非凸、非连续，数学性质不太好，使得式(6.29)不易直接求解。于是，人们通常用其他一些函数来代替 $\ell_{0/1}$，称为“替代损失”(surrogate loss)。替代损失函数一般具有较好的数学性质，如它们通常是凸的连续函数且是 $\ell_{0/1}$ 的上界。图 6.5 给出了三种常用的替代损失函数：

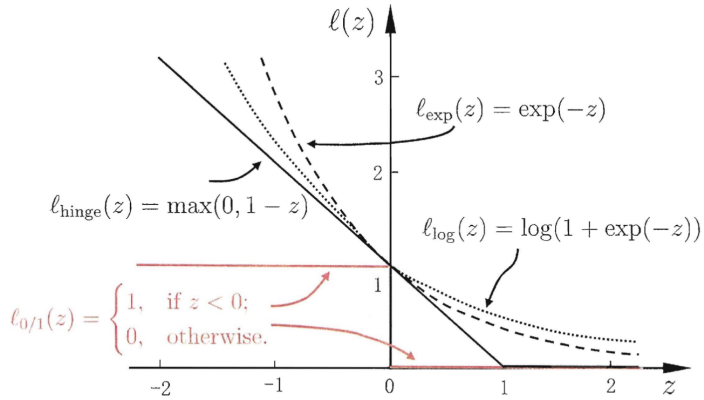


图 6.5 三种常见的替代损失函数：hinge 损失、指数损失、对率损失

$$\text{hinge 损失:} \quad \ell_{hinge}(z) = \max(0, 1-z); \tag{6.31}$$

$$\text{指数损失 (exponential loss):} \quad \ell_{exp}(z) = \exp(-z); \tag{6.32}$$

$$\text{对率损失 (logistic loss):} \quad \ell_{log}(z) = \log(1+\exp(-z)). \tag{6.33}$$

补充：对率损失函数 *(logistic loss)* 实质上是“对率函数”（*Logistic function*，即 *Sigmoid* 函数 $y = \frac{1}{1+e^{-z}}$）在负对数似然视角下的变形。它为 0/1 损失提供了一个平滑、可微且凸的代理函数，这使得原本难以求解的优化问题（式 *6.29*）转变为一个性质优良的优化问题，极大地便利了梯度下降等算法的应用。

小知识：*hinge* 损失与深度学习中著名的 *ReLU* 激活函数 $R(z) = \max(0, z)$ 在数学形式上互为“孪生”。事实上，$\ell_{hinge}(z) = ReLU(1-z)$。这种“取零最大化”的共同特征赋予了 *SVM* 与现代神经网络共同的底层基因——稀疏性（*Sparsity*）：当样本被正确分类且处于间隔带之外时，损失降为 *0*，这正是 *SVM* 模型仅由极少数“支持向量”决定的数学根源。

若采用 hinge 损失，则式(6.29)变成

$$\min_{\boldsymbol{w},b} \frac{1}{2}||\boldsymbol{w}||^2 + C\sum_{i=1}^{m} \max\left(0, 1 - y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b)\right) \tag{6.34}$$

引入“松弛变量”(slack variables) $\xi_i \geq 0$，可将式(6.34)重写为

$$\min_{\boldsymbol{w},b,\boldsymbol{\xi}} \frac{1}{2}||\boldsymbol{w}||^2 + C\sum_{i=1}^{m} \xi_i \tag{6.35}$$

$$\text{s.t. } y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1 - \xi_i,$$

$$\xi_i \geq 0, \quad i = 1, 2, \ldots, m.$$

这就是常用的“软间隔支持向量机”。

显然，式(6.35)中每个样本都有一个对应的松弛变量，用以表征该样本不满足约束(6.28)的程度。但是，与式(6.6)相似，这仍是一个二次规划问题。于是，类似式(6.8)，通过拉格朗日乘子法可得到式(6.35)的拉格朗日函数

$$L(\boldsymbol{w}, b, \boldsymbol{\alpha}, \boldsymbol{\xi}, \boldsymbol{\mu}) = \frac{1}{2}||\boldsymbol{w}||^2 + C\sum_{i=1}^{m}\xi_i + \sum_{i=1}^{m}\alpha_i\left(1 - \xi_i - y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b)\right) - \sum_{i=1}^{m}\mu_i\xi_i \tag{6.36}$$

其中 $\alpha_i \geq 0, \mu_i \geq 0$ 是拉格朗日乘子。

令 $L(\boldsymbol{w}, b, \boldsymbol{\alpha}, \boldsymbol{\xi}, \boldsymbol{\mu})$ 对 $\boldsymbol{w}, b, \xi_i$ 的偏导为零可得

$$\boldsymbol{w} = \sum_{i=1}^{m}\alpha_i y_i \boldsymbol{x}_i \tag{6.37}$$

$$0 = \sum_{i=1}^{m}\alpha_i y_i \tag{6.38}$$

$$C = \alpha_i + \mu_i \tag{6.39}$$

将式(6.37)至式(6.39)代入式(6.36)即可得到式(6.35)的对偶问题

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{m}\alpha_i - \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{m}\alpha_i\alpha_j y_i y_j \boldsymbol{x}_i^T\boldsymbol{x}_j \tag{6.40}$$

$$\text{s.t.} \quad \sum_{i=1}^{m}\alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C, \quad i = 1, 2, \ldots, m.$$

将式(6.40)与硬间隔下的对偶问题(6.11)对比可看出，两者唯一的差别就在于对偶变量的约束不同：前者是 $0 \leq \alpha_i \leq C$，后者是 $0 \leq \alpha_i$。于是，可采用 6.2 节中同样的算法求解式(6.40)；在引入核函数后能得到与式(6.24)同样的支持向量展式。

类似式(6.13)，对软间隔支持向量机，KKT 条件要求

$$\begin{cases} \alpha_i \geq 0, \quad \mu_i \geq 0 \\ y_i f(\boldsymbol{x}_i) - 1 + \xi_i \geq 0 \\ \alpha_i(y_i f(\boldsymbol{x}_i) - 1 + \xi_i) = 0 \\ \xi_i \geq 0, \quad \mu_i\xi_i = 0 \end{cases} \tag{6.41}$$

于是，对任意训练样本 $(\boldsymbol{x}_i, y_i)$，总有 $\alpha_i = 0$ 或 $y_i f(\boldsymbol{x}_i) = 1 - \xi_i$。若 $\alpha_i = 0$，则该样本不会对 $f(\boldsymbol{x})$ 有任何影响；若 $\alpha_i > 0$，则必有 $y_i f(\boldsymbol{x}_i) = 1 - \xi_i$，即该样本是支持向量：由式(6.39)可知，若 $\alpha_i < C$，则 $\mu_i > 0$，进而有 $\xi_i = 0$，即该样本恰在最大间隔边界上；若 $\alpha_i = C$，则有 $\mu_i = 0$，此时若 $\xi_i \leq 1$ 则该样本落在最大间隔内部，若 $\xi_i > 1$ 则该样本被错误分类。由此可看出，软间隔支持向量机的最终模型仅与支持向量有关，即通过采用 hinge 损失函数仍保持了稀疏性。

6.4.2 正则化

那么，能否对式(6.29)使用其他的替代损失函数呢？可以发现，如果使用对率损失函数 ℓ_{log} 来替代式(6.29)中的 0/1 损失函数，则几乎就得到了对率回归模型 (3.27)。实际上，支持向量机与对率回归的优化目标相近，通常情形下它们的性能也相当。

【形象化理解：正则化在解决什么问题？】
可以把模型训练理解成“既要把题做对，又不能把答案背死”。如果只顾着让训练样本上的误差尽量小，模型可能会把训练集中的偶然噪声也一并记住，从而在新样本上表现变差；而正则化做的事情，就是在“拟合训练数据”和“保持模型简洁”之间加一个平衡器。前者关心模型在已见样本上错得多不多，后者关心模型本身是不是过于复杂；二者折中之后，模型通常会有更好的泛化能力。

我们还可以把式(6.29)中的 0/1 损失函数换成别的替代损失函数以得到其他学习模型，这些模型的性质与所用的替代函数直接相关，但它们具有一个共性：优化目标中的第一项用来描述划分超平面的“间隔”大小，另一项用来表述训练集上的误差，可写为更一般的形式

$$\min_f \underbrace{\Omega(f)}_{\text{结构风险}} + \underbrace{C\sum_{i=1}^{m}\ell(f(\boldsymbol{x}_i), y_i)}_{\text{经验风险}} \tag{6.42}$$

其中 $\Omega(f)$ 称为“结构风险”(structural risk)，用于描述模型 f 的某些性质；第二项 $\sum_{i=1}^{m}\ell(f(\boldsymbol{x}_i), y_i)$ 称为“经验风险”(empirical risk)，用于描述模型与训练数据的契合程度；C 用于对二者进行折中。

从经验风险最小化的角度来看，$\Omega(f)$ 表述了我们希望获得具有何种性质的模型，例如希望模型复杂度较小。这为引入领域知识和用户意图提供了途径。

另一方面，这类约束信息有助于削减假设空间，从而降低仅仅最小化训练误差所带来的过拟合风险。从这个角度来说，式(6.42)称为“正则化”(regularization) 问题，$\Omega(f)$ 称为正则化项，C 则称为正则化常数。

L_p 范数 (norm) 是常用的正则化项。其中，L_2 范数 $||\boldsymbol{w}||_2^2$ 倾向于使 $\boldsymbol{w}$ 的分量取值尽量均衡，即非零分量个数尽量稠密；而 L_0 范数 $||\boldsymbol{w}||_0$ 和 L_1 范数 $||\boldsymbol{w}||_1$ 则倾向于使 $\boldsymbol{w}$ 的分量尽量稀疏，即非零分量个数尽量少。

【注记：不同 L_p 范数为何会导向不同性质？】

设参数向量 $\boldsymbol{w} = (w_1, w_2, \ldots, w_d)^T$，对 $p \geq 1$，L_p 范数的一般形式为

$$||\boldsymbol{w}||_p = \left(\sum_{i=1}^{d} |w_i|^p \right)^{1/p}.$$

其中，常见的几种情形可写为

$$||\boldsymbol{w}||_0 = \sum_{i=1}^{d} I(w_i \neq 0),$$

$$||\boldsymbol{w}||_1 = \sum_{i=1}^{d} |w_i|,$$

$$||\boldsymbol{w}||_2 = \left(\sum_{i=1}^{d} w_i^2 \right)^{1/2},$$

$$||\boldsymbol{w}||_\infty = \max_{1 \leq i \leq d} |w_i|.$$

其中，L_0 范数直接统计非零分量个数，因此最自然地对应“稀疏”这一目标；L_1 范数虽然仍是连续可优化的，但其等值线带有尖角，优化时更容易在某些坐标轴上取得最优点，因此常导出稀疏解；而 L_2 范数的等值线更圆滑，更倾向于让多个分量共同分担权重，因此通常得到较为稠密、较为均衡的解；L_∞ 范数则刻画分量绝对值中的最大者，强调对单个最大分量的控制。

6.5 支持向量回归

现在我们来考虑回归问题。给定训练样本 $D = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \ldots, (\boldsymbol{x}_m, y_m)\}, y_i \in \mathbb{R}$，希望学得一个形如式(6.7)的回归模型，使得 $f(\boldsymbol{x})$ 与 y 尽可能接近，$\boldsymbol{w}$ 和 b 是待确定的模型参数。

【形象化理解：SVR 在做什么？】

可以把 SVR 想成“给回归曲线套上一条厚度为 2ϵ 的保护带”。只要样本点落在这条带内，就认为预测“已经足够接近”，不再为它付出额外代价；只有落到带外的样本，才按偏离带边界的距离计算损失。于是，SVR 并不追求让每个样本都被精确穿过，而是追求得到一条尽量平滑、同时让大多数样本落入容忍带中的回归曲线。

对样本 $(\boldsymbol{x}, y)$，传统回归模型通常直接基于模型输出 $f(\boldsymbol{x})$ 与真实输出 y 之间的差别来计算损失，当且仅当 $f(\boldsymbol{x})$ 与 y 完全相同时，损失才为零。与此不同，支持向量回归 (Support Vector Regression，简称 SVR) 假设我们能容忍 $f(\boldsymbol{x})$ 与 y 之间最多有 ϵ 的偏差，即仅当 $f(\boldsymbol{x})$ 与 y 之间的差别绝对值大于 ϵ 时才计算损失。如图 6.6所示，这相当于以 $f(\boldsymbol{x})$ 为中心，构建了一个宽度为 2ϵ 的间隔带，若训练样本落入此间隔带，则认为是被预测正确的。

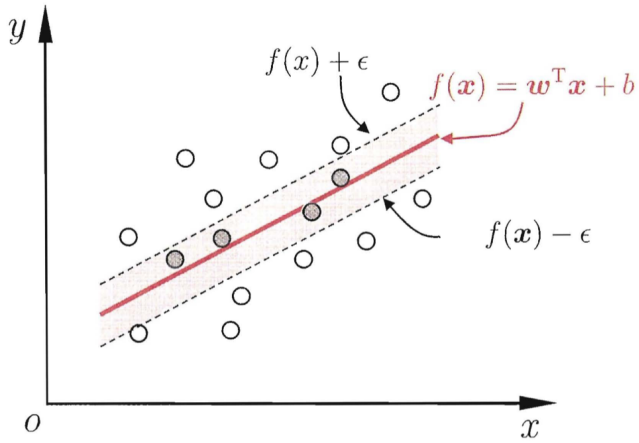


图 6.6 支持向量回归示意图。红色显示出 ϵ-间隔带，落入其中的样本不计算损失。

于是，SVR 问题可形式化为

$$\min_{\boldsymbol{w},b}\frac{1}{2}||\boldsymbol{w}||^2+C\sum_{i=1}^{m}\ell_\epsilon(f(\boldsymbol{x}_i)-y_i) \tag{6.43}$$

其中 C 为正则化常数，ℓ_ϵ 是图 6.7所示的 ϵ-不敏感损失 (ϵ-insensitive loss) 函数

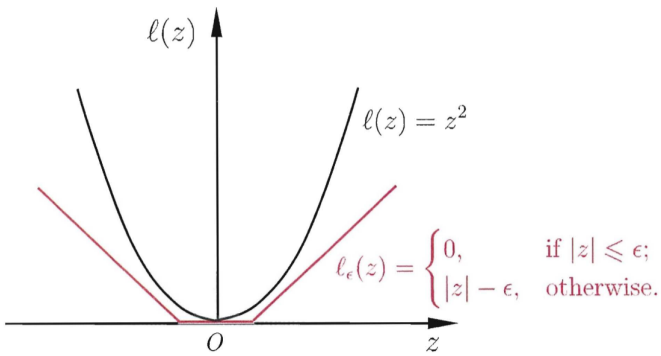


图 6.7 ϵ-不敏感损失函数

$$\ell_\epsilon(z)=\begin{cases}0, & \text{if } |z|\leq\epsilon\\ |z|-\epsilon, & \text{otherwise}\end{cases} \tag{6.44}$$

引入松弛变量 ξ_i 和 $\hat{\xi}_i$，可将式(6.43)重写为

$$\min_{\boldsymbol{w},b,\boldsymbol{\xi},\hat{\boldsymbol{\xi}}}\frac{1}{2}||\boldsymbol{w}||^2+C\sum_{i=1}^{m}(\xi_i+\hat{\xi}_i) \tag{6.45}$$

$$\text{s.t. } f(\boldsymbol{x}_i)-y_i\leq\epsilon+\xi_i,$$

$$y_i-f(\boldsymbol{x}_i)\leq\epsilon+\hat{\xi}_i,$$

$$\xi_i \geq 0, \hat{\xi}_i \geq 0, \quad i = 1, 2, \ldots, m.$$

类似式(6.36)，通过引入拉格朗日乘子 $\mu_i \geq 0$，$\hat{\mu}_i \geq 0$，$\alpha_i \geq 0$，$\hat{\alpha}_i \geq 0$，由拉格朗日乘子法可得到式(6.45)的拉格朗日函数

$$\begin{aligned} L(\boldsymbol{w}, b, \boldsymbol{\alpha}, \hat{\boldsymbol{\alpha}}, \boldsymbol{\xi}, \hat{\boldsymbol{\xi}}, \boldsymbol{\mu}, \hat{\boldsymbol{\mu}}) =& \frac{1}{2}||\boldsymbol{w}||^2 + C\sum_{i=1}^{m}(\xi_i + \hat{\xi}_i) - \sum_{i=1}^{m}\mu_i\xi_i - \sum_{i=1}^{m}\hat{\mu}_i\hat{\xi}_i \\ &+ \sum_{i=1}^{m}\alpha_i(f(\boldsymbol{x}_i) - y_i - \epsilon - \xi_i) + \sum_{i=1}^{m}\hat{\alpha}_i(y_i - f(\boldsymbol{x}_i) - \epsilon - \hat{\xi}_i) \end{aligned} \tag{6.46}$$

将式(6.7)中的 $f(\boldsymbol{x})$ 代入，再令 L 对 $\boldsymbol{w}, b, \xi_i$ 和 $\hat{\xi}_i$ 的偏导为零可得

$$\boldsymbol{w} = \sum_{i=1}^{m}(\hat{\alpha}_i - \alpha_i)\boldsymbol{x}_i \tag{6.47}$$

$$0 = \sum_{i=1}^{m}(\hat{\alpha}_i - \alpha_i) \tag{6.48}$$

$$C = \alpha_i + \mu_i \tag{6.49}$$

$$C = \hat{\alpha}_i + \hat{\mu}_i \tag{6.50}$$

将式(6.47)至式(6.50)代入式(6.46)，即可得到 SVR 的对偶问题

$$\max_{\boldsymbol{\alpha}, \hat{\boldsymbol{\alpha}}} \sum_{i=1}^{m} y_i(\hat{\alpha}_i - \alpha_i) - \epsilon(\hat{\alpha}_i + \alpha_i) - \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{m}(\hat{\alpha}_i - \alpha_i)(\hat{\alpha}_j - \alpha_j)\boldsymbol{x}_i^T\boldsymbol{x}_j \tag{6.51}$$

$$\text{s.t.} \quad \sum_{i=1}^{m}(\hat{\alpha}_i - \alpha_i) = 0, \quad 0 \leq \alpha_i, \hat{\alpha}_i \leq C.$$

上述过程中需满足 KKT 条件，即要求

$$\begin{cases} \alpha_i(f(\boldsymbol{x}_i) - y_i - \epsilon - \xi_i) = 0 \\ \hat{\alpha}_i(y_i - f(\boldsymbol{x}_i) - \epsilon - \hat{\xi}_i) = 0 \\ \alpha_i\hat{\alpha}_i = 0, \quad \xi_i\hat{\xi}_i = 0 \\ (C - \alpha_i)\xi_i = 0, \quad (C - \hat{\alpha}_i)\hat{\xi}_i = 0 \end{cases} \tag{6.52}$$

可以看出，当且仅当 $f(\boldsymbol{x}_i) - y_i - \epsilon - \xi_i = 0$ 时 α_i 能取非零值，当且仅当 $y_i - f(\boldsymbol{x}_i) - \epsilon - \hat{\xi}_i = 0$ 时 $\hat{\alpha}_i$ 能取非零值。换言之，仅当样本 $(\boldsymbol{x}_i, y_i)$ 不落入 ϵ-间隔带中，相应的 α_i 和 $\hat{\alpha}_i$ 才能取非零值。此外，约束不能同时成立，因此 α_i 和 $\hat{\alpha}_i$ 中至少有一个为零。

将式(6.47)代入式(6.7)，则 SVR 的解形如

$$f(\boldsymbol{x}) = \sum_{i=1}^{m}(\hat{\alpha}_i - \alpha_i)\boldsymbol{x}_i^T\boldsymbol{x} + b \tag{6.53}$$

能使式(6.53)中的 $(\hat{\alpha}_i - \alpha_i) \neq 0$ 的样本即为 SVR 的支持向量，它们必落在 ϵ-间隔带之外。显然，SVR 的支持向量仅是训练样本的一部分，即其解仍具有稀疏性。

由 KKT 条件(6.52)可看出，对每个样本 $(\boldsymbol{x}_i, y_i)$ 都有 $(C - \alpha_i)\xi_i = 0$ 且 $\alpha_i(f(\boldsymbol{x}_i) - y_i - \epsilon - \xi_i) = 0$。于是，在得到 α_i 后，若 $0 < \alpha_i < C$，则必有 $\xi_i = 0$，进而有

$$b = y_i + \epsilon - \sum_{j=1}^{m} (\hat{\alpha}_j - \alpha_j) \boldsymbol{x}_j^T \boldsymbol{x}_i \tag{6.54}$$

因此，在求解式(6.51)得到 α_i 后，理论上来说，可任意选取满足 $0 < \alpha_i < C$ 的样本通过式(6.54)求得 b。实践中常采用一种更鲁棒的办法：选取多个（或所有）满足条件 $0 < \alpha_i < C$ 的样本求解 b 后取平均值。

若考虑特征映射形式(6.19)，则相应的，式(6.47)将形如

$$\boldsymbol{w} = \sum_{i=1}^{m} (\hat{\alpha}_i - \alpha_i) \phi(\boldsymbol{x}_i) \tag{6.55}$$

将式(6.55)代入式(6.19)，则 SVR 可表示为

$$f(\boldsymbol{x}) = \sum_{i=1}^{m} (\hat{\alpha}_i - \alpha_i) \kappa(\boldsymbol{x}, \boldsymbol{x}_i) + b \tag{6.56}$$

其中 $\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \phi(\boldsymbol{x}_i)^T \phi(\boldsymbol{x}_j)$ 为核函数。

6.6 核方法

【注记：核方法到底在解决什么问题?】

许多经典学习方法本质上都是**线性方法**：它们擅长在“直的世界”里分类、回归或做投影；但现实数据的结构往往是弯曲的、环状的、纠缠的，在原始空间里并不容易被线性方法处理。核方法要解决的问题，就是：**如何不推翻这些线性方法的骨架，却让它们具有处理非线性问题的能力。**

一种形象的理解是：线性学习器好比一把直尺，在平面上只能画直线；如果数据本身是弯的，那么问题不一定出在直尺上，而可能出在“空间太低”。若把数据映射到更高维空间，它们也许就会变得线性可分、线性可回归、线性可投影。

但真正把样本显式映射到高维空间往往代价很大，甚至那个空间可能是无限维的。核方法的巧妙之处在于：**不显式进行高维映射，而是直接计算样本在高维空间中的内积结果。**也就是说，它并不真的“搬到高维空间去算”，而是通过核函数 $\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j)$ 直接告诉我们：这两个样本若映射到高维后，它们的内积会是多少。

因此，核方法的基本套路是：**先找出一个线性方法中依赖内积的部分，再把普通内积替换成核函数。**这样，原本的线性方法就被“核化”为非线性方法，而它的求解框架通常仍能保留下来。

6.6.1 表示定理

回顾式(6.24)和(6.56)可发现，给定训练样本 $\{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \ldots, (\boldsymbol{x}_m, y_m)\}$，若不考虑偏移项 b，则无论 SVM 还是 SVR，学得的模型总能表示成核函数 $\kappa(\boldsymbol{x}, \boldsymbol{x}_i)$ 的线性组合。不仅如此，事实上我们有下面这个称为“表示定理”(representer theorem) 的更一般的结论：

定理 6.2 (表示定理) 令 $\mathcal{H}$ 为核函数 κ 对应的再生核希尔伯特空间，$||h||_{\mathcal{H}}$ 表示 $\mathcal{H}$ 空间中关于 h 的范数，对于任意单调递增函数 $\Omega : [0, \infty) \mapsto \mathbb{R}$ 和任意非负损失函数 $\ell : \mathbb{R}^m \to [0, \infty]$，优化问题

$$\min_{h \in \mathcal{H}} F(h) = \Omega(||h||_{\mathcal{H}}) + \ell(h(\boldsymbol{x}_1), h(\boldsymbol{x}_2), \ldots, h(\boldsymbol{x}_m)) \tag{6.57}$$

的解总可写为

$$h^*(\boldsymbol{x}) = \sum_{i=1}^{m} \alpha_i \kappa(\boldsymbol{x}, \boldsymbol{x}_i) \tag{6.58}$$

表示定理对损失函数没有限制，对正则化项 Ω 仅要求单调递增，甚至不要求 Ω 是凸函数，意味着对于一般的损失函数和正则化项，优化问题(6.57)的最优解 $h^*(\boldsymbol{x})$ 都可表示为核函数 $\kappa(\boldsymbol{x}, \boldsymbol{x}_i)$ 的线性组合；这显示出核函数的巨大威力。

6.6.2 KLDA 示例

人们发展出一系列基于核函数的学习方法，统称为“核方法”(kernel methods)。最常见的，是通过“核化”(即引入核函数) 来将线性学习器拓展为非线性学习器。下面我们以线性判别分析为例来演示如何通过核化来对其进行非线性拓展，从而得到“核线性判别分析”(Kernelized Linear Discriminant Analysis，简称 KLDA)。

【形象化理解：KLDA（核线性判别分析）】

1. 任务目标（它想干什么?）

常规的 LDA 就像是用一把直尺在平面的相片上划线，试图把不同颜色的人群“切”开。但如果一群人围成一圈（数据呈环状或非线性分布），直尺就束手无策了。

KLDA 的目标就是要**打破这把直尺的限制**，它想找出一把“魔法变向刀”（非线性的投影面），去切开那些像迷宫一样复杂交织在一起的异类分类样本，同时依然死死守住 LDA **“同类紧密相拥，异类远远避开”**的核心灵魂。

2. 方法原理（它是怎么做到的?）

KLDA 的手段非常巧妙，堪称**“升维打击”与“降维计算”的完美结合：**

- **虚拟升维（思想上）：**它在设想中把所有处于平面死结状态的样本点，“砰”地一声发射到极高维（甚至是无限维）的太空中（特征映射 $\phi(\boldsymbol{x})$）。在这个极其宽广的空间里，原本纠缠的数据彻底散开，只需要一张平整的白纸（高维线性平面）就能把不同类别切开了。

- **偷天换日（数学上）：**但是，真的飞去无限维太空做计算，无论多少台超级计算机都会算到死机（维数灾难）。万幸的是，通过**表示定理**，数学家发现高维空间里的那张“白纸”（权重向量 $\boldsymbol{w}$），其实完全可以由地上原始样本的复杂互动（映射后的内积组合）来唯一确定。

- **核技巧降临（计算上）：**既然一切只和高维内积有关，那我们干脆**不用起飞**了！直接使用**核函数**（就像一个次元传送门），在低维大地上原封不动地输入两个样本，它就能瞬间送回这两个样本在高空相撞（内积）后的结果。于是，高维天界难以求解的任务，被彻底“降维”成了在凡间求解简单参数矩阵（即式 6.70 中的 $\boldsymbol{\alpha}$）的数字游戏。

我们先假设可通过某种映射 $\phi : \mathcal{X} \mapsto \mathcal{F}$ 将样本映射到一个特征空间 $\mathcal{F}$，然后在 $\mathcal{F}$ 中执行线性判别分析，以求得

$$h(\boldsymbol{x}) = \boldsymbol{w}^T \phi(\boldsymbol{x}) \tag{6.59}$$

类似于式 (3.35)，KLDA 的学习目标是

$$\max_{\boldsymbol{w}} J(\boldsymbol{w}) = \frac{\boldsymbol{w}^T S_b^\phi \boldsymbol{w}}{\boldsymbol{w}^T S_w^\phi \boldsymbol{w}} \tag{6.60}$$

其中 S_b^ϕ 和 S_w^ϕ 分别为训练样本在特征空间 $\mathcal{F}$ 中的类间散度矩阵和类内散度矩阵。令 X_i 表示第 $i \in \{0, 1\}$ 类样本的集合，其样本数为 m_i；总样本数 $m = m_0 + m_1$。第 i 类样本在特征空间 $\mathcal{F}$ 中的均值为

$$\boldsymbol{\mu}_i^\phi = \frac{1}{m_i} \sum_{\boldsymbol{x} \in X_i} \phi(\boldsymbol{x}) \tag{6.61}$$

两个散度矩阵分别为

$$S_b^\phi = (\boldsymbol{\mu}_0^\phi - \boldsymbol{\mu}_1^\phi)(\boldsymbol{\mu}_0^\phi - \boldsymbol{\mu}_1^\phi)^T \tag{6.62}$$

$$S_w^\phi = \sum_{i=0}^{1} \sum_{\boldsymbol{x} \in X_i} (\phi(\boldsymbol{x}) - \boldsymbol{\mu}_i^\phi)(\phi(\boldsymbol{x}) - \boldsymbol{\mu}_i^\phi)^T \tag{6.63}$$

【注记：什么是散度矩阵（Scatter Matrix）?】

散度矩阵是“方差”在多维空间里的升维版本。假设我们在二维平面里统计一群人的“身高”和“体重”，把每个人的坐标写成列向量 $\boldsymbol{x} = [\text{身高}, \text{体重}]^T$，并算出平均坐标 $\boldsymbol{\mu} = [\text{平均身高}, \text{平均体重}]^T$。

计算多维散度矩阵的动作是：让每个人的向量减去平均坐标得到偏差向量 $(\boldsymbol{x} - \boldsymbol{\mu})$，然后让这个列向量乘上它自己的行向量，也就是计算外积 $(\boldsymbol{x} - \boldsymbol{\mu})(\boldsymbol{x} - \boldsymbol{\mu})^T$。这会为每个人生成一个小的方块矩阵。把所有人的小矩阵加起来，就得到了全局的散度矩阵 S。它是二维的，就是一个 2×2 的方块：

$$\begin{bmatrix} \text{身高的纯方差} & \text{身高和体重的联动方差（协方差）} \\ \text{体重和身高的联动方差（协方差）} & \text{体重的纯方差} \end{bmatrix}$$

通过这个矩阵，我们就不仅知道了大家高矮胖瘦的独立差异，还完美掌握了“个子高的人往往体重也大”这种倾斜联动的全貌。

散度矩阵与协方差矩阵有什么区别？

一句话：散度矩阵就是**“没有求平均值”**的协方差矩阵。散度矩阵是“总和的狂欢”，它只是粗暴地把每个样本偏离中心的平方值**加起来**，这意味着测 100 个人的数值会是测 10 个人的十倍；而协方差矩阵为了剥离人数影响，在最后除了人数以计算“平均每个人的发散程度”。而在公式 6.60 里，由于分子和分母都是散度矩阵，如果大家都除以人数，常数项就会直接上下约掉、不影响我们寻找使得比值最大的直尺方向 $\boldsymbol{w}$。所以，数学家为了计算省事，就直接弃去除了除法步骤，选用了原始的散度矩阵。

【注记：如何拆解理解公式 6.60 的目标?】
在看到上述分子与分母时，我们可以通过“投影降维”的视角将其化繁为简：

- **分子的含义：两队中心离多远？($\boldsymbol{w}^T S_b^\phi \boldsymbol{w}$)**
  式 6.62 定义的 S_b^ϕ（类间散度矩阵，b 代表 between）记录了“两队大本营中心”在高维空间的物理差距。当我们用代表直尺方向的向量 $\boldsymbol{w}^T$ 和 $\boldsymbol{w}$ 从左右“夹击”这个矩阵时，它在数学上完全等价于：**先把两队中心投影到一维直尺上，再计算投影点的距离平方**。这就将高维的矩阵压缩成了一个表示“隔离度”的一维数字指标。

- **分母的含义：自己人抱多紧？($\boldsymbol{w}^T S_w^\phi \boldsymbol{w}$)**
  式 6.63 定义的 S_w^ϕ（类内散度矩阵，w 代表 within）记录了各自队员离自家中心的离散程度。同理，经过左右夹击后的 $\boldsymbol{w}^T S_w^\phi \boldsymbol{w}$，坍缩成了另一项一维数字指标：**全员投影到一维尺子后，各个队伍自身的方差总和**。这个数字越小，说明大家在影子刻度上紧锁在一起，毫不松散。

核心总结：公式 6.60 的本质就是一个求最优拍照视角的过程，它所追求的那把高维直尺 $\boldsymbol{w}$，就是为了实现核心准则——**让分子（异类群的投影中心相距甚远）尽可能大，同时让分母（同属一类的投影点紧紧相拥）尽可能小。**

通常我们难以知道映射 ϕ 的具体形式，因此使用核函数 $\kappa(\boldsymbol{x}, \boldsymbol{x}_i) = \phi(\boldsymbol{x}_i)^T \phi(\boldsymbol{x})$ 来隐式地表达这个映射和特征空间 $\mathcal{F}$。把 $J(\boldsymbol{w})$ 作为式(6.57)中的损失函数 ℓ，再令 $\Omega \equiv 0$，由表示定理，函数 $h(\boldsymbol{x})$ 可写为

$$h(\boldsymbol{x}) = \sum_{i=1}^{m} \alpha_i \kappa(\boldsymbol{x}, \boldsymbol{x}_i) \tag{6.64}$$

于是由式(6.59)可得

$$\boldsymbol{w} = \sum_{i=1}^{m} \alpha_i \phi(\boldsymbol{x}_i) \tag{6.65}$$

令 $K \in \mathbb{R}^{m \times m}$ 为核函数 κ 所对应的核矩阵，$(K)_{ij} = \kappa(\boldsymbol{x}_i, \boldsymbol{x}_j)$。令 $\mathbf{1}_i \in \{1, 0\}^{m \times 1}$ 为第 i 类样本的指示向量，即 $\mathbf{1}_i$ 的第 j 个分量为 1 当且仅当 $\boldsymbol{x}_j \in X_i$，否则 $\mathbf{1}_i$ 的第 j 个分量为 0。再令

$$\boldsymbol{\beta}_0 = \frac{1}{m_0} K \mathbf{1}_0 \tag{6.66}$$

$$\boldsymbol{\beta}_1 = \frac{1}{m_1} K \mathbf{1}_1 \tag{6.67}$$

$$M = (\boldsymbol{\beta}_0 - \boldsymbol{\beta}_1)(\boldsymbol{\beta}_0 - \boldsymbol{\beta}_1)^T \tag{6.68}$$

$$N = KK^T - \sum_{i=0}^{1} m_i \boldsymbol{\beta}_i \boldsymbol{\beta}_i^T \tag{6.69}$$

于是，式(6.60)等价为

$$\max_{\boldsymbol{\alpha}} J(\boldsymbol{\alpha}) = \frac{\boldsymbol{\alpha}^T M \boldsymbol{\alpha}}{\boldsymbol{\alpha}^T N \boldsymbol{\alpha}} \tag{6.70}$$

显然，使用线性判别分析求解方法即可得到 $\boldsymbol{\alpha}$，进而可由式(6.64)得到投影函数 $h(\boldsymbol{x})$。